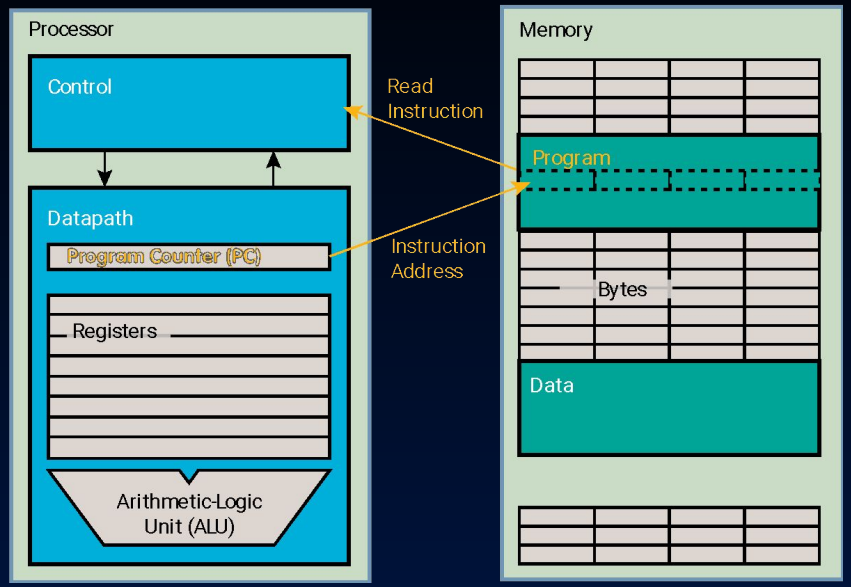
CS61C

Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)

Assistant
Teaching Professor
Lisa Yan

RISC-V Procedures

Review: RISC-V Control Flow



Instruction	Name/description
`beq rs1 rs2 Label`	Branch if EQual
`bne rs1 rs2 Label`	Branch if Not Equal
`blt rs1 rs2 Label`	Branch if Less Than (signed) (rs1 < rs2)
`bge rs1 rs2 Label`	Branch if Greater or Equal (signed) (rs1 >= rs2)
`bltu rs1 rs2 Label`	Branch if Less Than (unsigned)
`bgeu rs1 rs2 Label`	Branch if Greater Than or Equal (unsigned)
`j Label`	Unconditional jump

Agenda

Function Calls

- Function Calls
- Stack Frames
- RISC-V Calling Convention
- [Venus] Recursive Example
- [Reference] Summary, Other Registers
- [Extra] Recursive Example 2

`jal`: Call a Function, `jr`: Return from a Function

- Jump and link

```
jal rd Label
R[rd] = PC + 4
PC = <address of Label>
```

Really should be:
"**Link** and **Jump**"!

- Jump register (pseudo)

```
jr rs1
PC = R[rs1]
```

Common instruction pair:

- `jal ra FunctionName`
- `jr ra`

`ra` (**return address**) is a register name for register x1.

jal, jr in action

```
0x 0  main:
          li a0 4
   4      jal ra factorial
   8      addi a1 a0 0
  …       …
  20      li a0 3
  24      jal ra factorial
  28      addi a1 a0 0
  …       …
  48  factorial:
          addi t0 x0 1
  4c      addi t1 x0 1
  50  loop:
          blt a0 t0 done
  54      mul t1 t1 t0
  58      addi t0 t0 1
  5c      j loop
  60  done:
          add a0 t1 x0
  64      jr ra
```

Function call:
`jal @ 0x4`

pc	~~0x4~~	0x48
ra	~~0xABCD ABCD~~	0x8

Function return:
`jr @ 0x64`

pc	~~0x64~~	0x8
ra	0x8	

jal, jr in action

```
0x 0  main:
          li a0 4
     4    jal ra factorial
     8    addi a1 a0 0
     …    …
    20    li a0 3
    24    jal ra factorial
    28    addi a1 a0 0
     …    …
    48  factorial:
          addi t0 x0 1
    4c    addi t1 x0 1
    50  loop:
          blt a0 t0 done
    54    mul t1 t1 t0
    58    addi t0 t0 1
    5c    j loop
    60  done:
          add a0 t1 x0
    64    jr ra
```

Function call:
jal @ 0x24

pc	~~0x24~~	0x48
ra	~~0xA~~	0x28

Function return:
jr @ 0x64

pc	~~0x64~~	0x28
ra	0x28	

Four common jump instructions/pseudoinstructions

- **J**ump **a**nd **l**ink
  ```
  jal rd Label
  R[rd] = PC + 4
  PC = <address of Label>
  ```
- **J**ump **r**egister (pseudo)
  ```
  jr rs1
  PC = R[rs1]
  ```
  - Translation: `jalr x0 rs1 0`
- **J**ump **a**nd **l**ink **r**egister
  ```
  jalr rd rs1 imm
  R[rd] = PC + 4
  PC = R[rs1] + imm
  ```
- **J**ump (pseudo)
  ```
  j Label
  PC = <address of Label>
  ```
  - Translation: `jal x0 Label`

Four common jump instructions/pseudoinstructions

- Jump and link
  `jal rd Label`
- Jump register (pseudo)
  `jr rs1`
  - Translation: `jalr x0 rs1 0`
- Jump and link register
  `jalr rd rs1 imm`
- Jump (pseudo)
  `j Label`
  - Translation: `jal x0 Label`

Read more in course notes

RISC-V Register Conventions, Part 1 of 3

- Registers faster than memory, so use them!
  - Register names provide **conventions** for calling functions, passing in arguments, and returning from functions.
- `a0–a7` (`x10–x17`):
  - eight argument registers to pass parameters
  - two return values (`a0–a1`), though we usually use `a0`
- `ra` (`x1`):
  - one return address register to return to the point of origin
- More later: stack pointer, saved registers, temporary registers

Different functions share the **same set of registers** and use them with the **same calling conventions**.

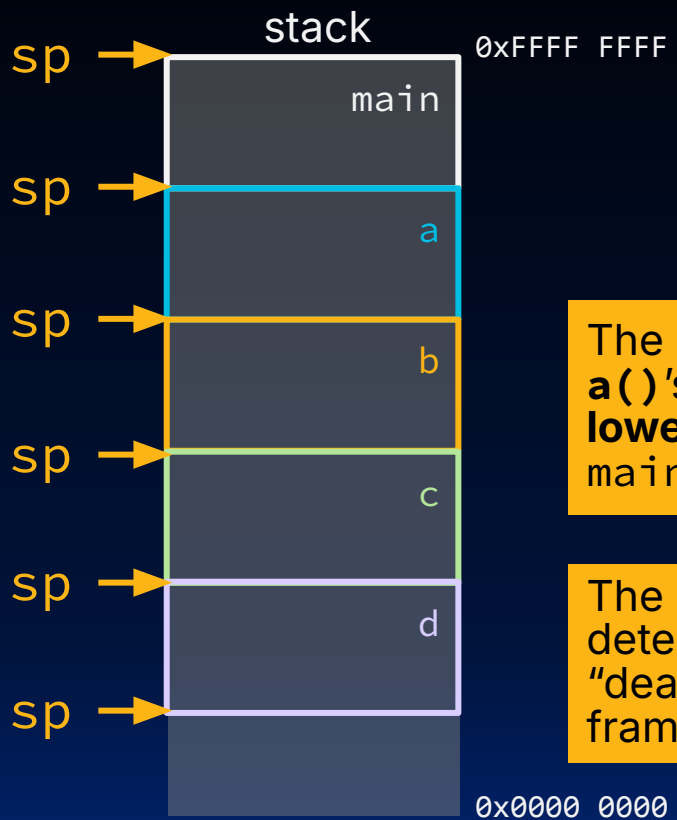
Agenda

Stack Frames

- Function Calls
- Stack Frames
- RISC-V Calling Convention
- [Venus] Recursive Example
- [Reference] Summary, Other Registers
- [Extra] Recursive Example 2

[Review] C Stack Frames

```
int main ()
{ a(0);  …
}
void a (int m)
{ b(1);
}
void b (int n)
{ c(2);
}
void c (int o)
{ d(3);
}
void d (int p)
{
}
```



The stack **grows downward; a()**'s local variables have **lower byte addresses** than `main()`'s, and so on.

The **stack pointer** (`sp`) is what determines "allocating" and "deallocating/freeing" stack frames!

[animate this slide]

RISC-V Stack

- RISC-V stack frames (mostly) operate like C stack frames.
  - Stack pointer register: `sp` (`x2`)
  - A function can choose to use a stack frame by manipulating `sp`:
    - After a function is called, **create** stack frame/**decrement** `sp`.
    - When function wraps up, **delete** stack frame/**increment** `sp`.

- Stack frame can include:
  - Space for local variables (e.g., stack arrays)
  - Return "instruction" address (why?)
  - Parameters (arguments) beyond the 8 argument registers

Not all RISC-V functions need stack frames. Some functions only need registers! (see earlier example)

Fundamental Steps in Calling a Function

1. Set up arguments

   Put arguments in `a0`, ..., `a7`, etc.

2. Transfer control to function

   `jal ra fnLabel`

3. Prologue

   Acquire (local) storage resources needed for function: stack, registers

4. Perform desired function task

5. Epilogue

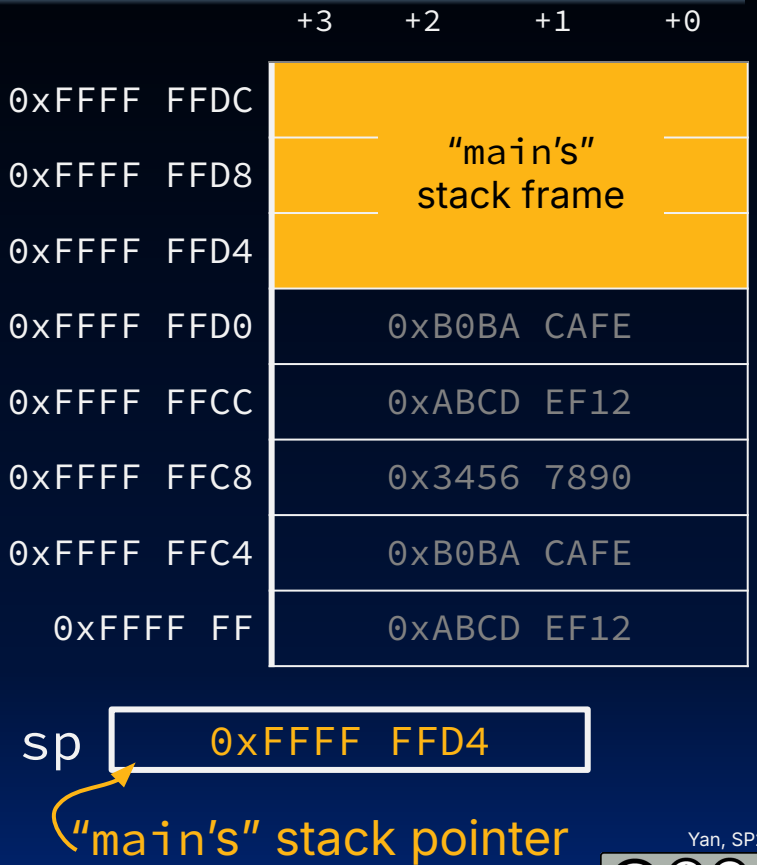
   - Put return value `a0` (or `a1`)
   - Restore any registers used
   - Release local storage on stack

6. Return control to point of origin

   `jr ra`

The RISC-V Stack Also Grows Down

```
0x 0   main:
   4       addi sp sp -12
   …       …
   …       # x calls fooA
   …       …
  20       addi sp sp 12
  24       j exit

  28   fooA:
           addi sp sp -8
           …
  50       addi sp sp 8
  54       jr ra
   …       …

  70   exit:
   …       …
```

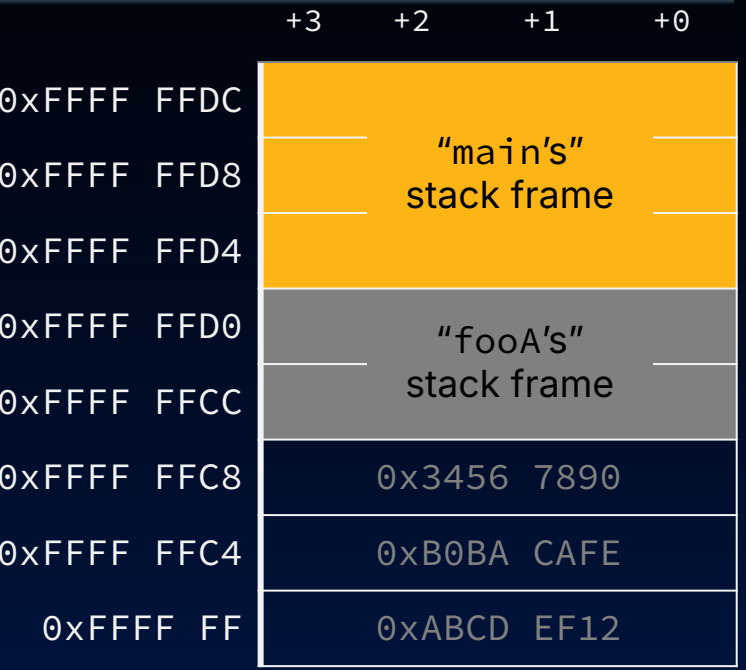


The RISC-V Stack Also Grows Down

```
0x 0  main:
   4      addi sp sp -12
  …       …
  …       # x calls fooA
  …       …
  20      addi sp sp 12
  24      j exit

  28  fooA:
      addi sp sp -8
  …   …
  50  addi sp sp 8
  54  jr ra
  …   …

  70  exit:
  …   …
```

The RISC-V Stack Also Grows Down

```
0x 0   main:
   4       addi sp sp -12
   …       …
   …       # x calls fooA
   …       …
  20       addi sp sp 12
  24       j exit

  28   fooA:
           addi sp sp -8
           …
  50       addi sp sp 8
  54       jr ra
   …       …

  70   exit:
   …       …
```

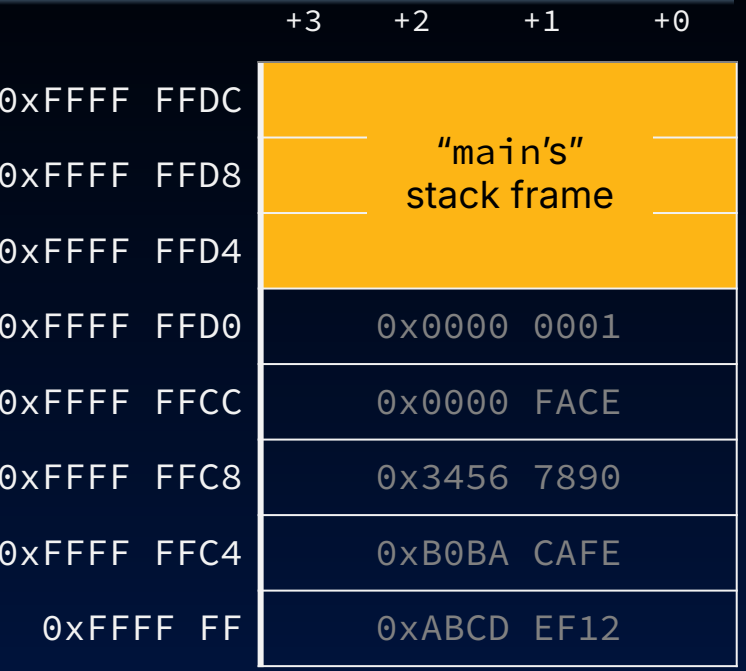


sp 0xFFFF FFD4

Restore caller's (here, main's) stack pointer before "returning"

The RISC-V Stack Also Grows Down

```
0x 0   main:
   4       addi sp sp -12
   …       …
   …       # x calls fooA
   …       …
  20       addi sp sp 12
  24       j exit

  28   fooA:
           addi sp sp -8
           …
  50       addi sp sp 8
  54       jr ra
   …       …

  70   exit:
   …       …
```



When main "has control," it sees the right stack pointer

RISC-V Register Conventions, Part 2 of 3

- Registers faster than memory, so use them!
  - Use stack frames sparingly!
- `a0–a7` (`x10–x17`):
  - eight argument registers to pass parameters
  - two return values (`a0–a1`), though we usually use `a0`
- `ra` (`x1`):
  - one return address register to return to the point of origin
- `sp` (`x2`):
  - stack pointer has address of bottom of stack (current stack frame)

Different functions share the **same set of registers** and use them with the **same calling conventions**.

RISC-V Register Conventions, Part 3 of 3

- Registers faster than memory, so use them!
  - Use stack frames sparingly!
- `a0-a7` (`x10-x17`):
  - eight argument registers to pass parameters
  - two return values (`a0-a1`), though we usually use `a0`
- `ra` (`x1`):
  - one return address register to return to the point of origin
- `sp` (`x2`):
  - stack pointer has address of bottom of stack (current stack frame)
- `s0-s11` (`x8,x9,x18-x27`): Saved registers
- `t0-t6` (`x5-x7,x28-x31`): Temporary registers

Different functions share the **same set of registers** and use them with the **same calling conventions**.

} ?????

Agenda

RISC-V Calling Convention

- Function Calls
- Stack Frames
- RISC-V Calling Convention
- [Venus] Recursive Example
- [Reference] Summary, Other Registers
- [Extra] Recursive Example 2

❌ Clobbered Registers with Nested Function Calls

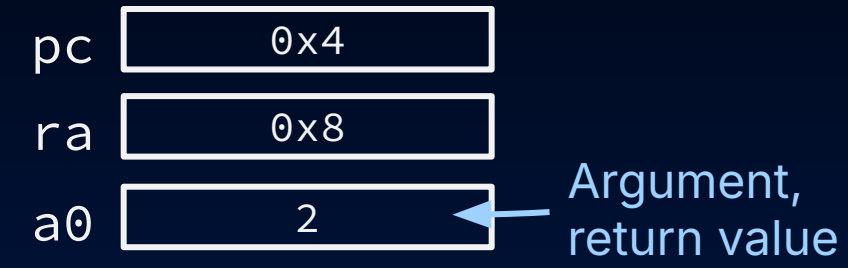
- Suppose we implement a **recursive** factorial function.

Below, what is the issue with `factorial(2)`?

```
// assume non-negative numbers
int factorial(int n) {
  if (n == 1) return 1;
  int a = n*factorial(n-1);
  return a;
}
```

Caller `factorial(2)`'s perspective:

1. Prepare for **callee** `factorial(1)` by passing in new argument, `1`
2. Pass control to **callee** `factorial(1)`
3. Multiply **callee**'s return value by multiplying by original argument, `2`
4. Return result



Because all functions use the same RISC-V **calling convention**, register values can get **clobbered** with nested function calls.

Register Calling Conventions

- **CalleR**: the calling function
- **CalleE**: the function being called
- When **callee** returns from executing, the **caller** needs to know:
  - Which registers may have changed, and
  - Which are guaranteed to be unchanged.
- Register **calling conventions**: A set of generally accepted rules as to which registers will be unchanged after a procedure call (`jal`) and which may be changed.
- To reduce expensive loads and stores from spilling and restoring registers, RISC-V function-calling convention divides registers into two categories:
  - **Not** preserved across function call
  - **Are** preserved across function call (if the **callee** changed any registers, it restored original values before returning)

Caller-saved vs. Callee-saved registers

- **Caller**-saved: *(if data in them)*
  - Return address, `ra`
  - Temporary registers, `t0-t6`
  - Arguments/retvals, `a0-a7`
- May **not** be preserved across function call!
- **Caller** is responsible for
  - Saving these register values **for itself before calling a callee**

- **Callee**-saved: *(if it uses them)*
  - Stack pointer, `sp`
  - Saved registers, `s0-s11`
- Should be preserved across function call!
- **Callee** is responsible for
  - Saving/restoring these registers values in the **function prologue**, and **epilogue**, respectively
  - Allocating/deallocating the **stack frame.**

Caller: To save register values, save to `s0-s11` or to your stack frame.

Callee: To save register values, make space in your stack frame.

Fundamental Steps in Calling a Function: CALLER

1. Set up for callee
   - Save caller-saved registers if needed, either by copying to `s0-s11` or to stack
   - Put arguments in `a0`, ..., `a7`, stack, etc.
2. Transfer control to callee
   - `jal ra fnLabel`
3. Prologue
4. Perform desired function task
5. Epilogue
6. Return control to point of origin
   - `jr ra # callee`
   - Restore caller-saved registers if needed

Fundamental Steps in Calling a Function: CALLEE

1. Set up for callee
2. Transfer control to callee
3. Prologue

- Push stack frame/decrement `sp`
- Save callee-saved regs if will use:
  - `s0-s11`
- Save ra if calling subroutine
- Save place for local non-register variables (e.g., stack arrays)

4. Perform desired function task
5. Epilogue

- Put return value `a0` (and `a1` if nec)
- Restore any callee-saved registers used, restore `ra` if called subroutine
- Pop stack frame/increment `sp`

6. Return control to point of origin

```
jr ra # callee
```

Restore caller-saved registers if needed

Agenda

[Venus] Recursive Example

- Function Calls
- Stack Frames
- RISC-V Calling Convention
- [Venus] Recursive Example
- [Reference] Summary, Other Registers
- [Extra] Recursive Example 2

(1/7) Recursive Factorial

```
0x 0  main:
        li a0 3
   4    jal ra factorial
        …
  48    factorial:
        addi sp sp -8          } Prologue
  4c    sw ra 0(sp)            }
  50    sw s0 4(sp)            }
  54    mv s0 a0
  58    li t0 1
  5c    bne s0 t0 recurse
  60    li a0 1
  64    j epilogue
  68    recurse:
        addi a0 s0 -1
  6c    jal ra factorial
  70    mul a0 s0 a0
  74    epilogue:
        lw ra 0(sp)            } Epilogue
  78    lw s0 4(sp)            }
  7c    addi sp sp 8           }
  80    jr ra
```

1. Prologue:
   a. How large is the stack frame?
   b. Which registers get saved to the stack?
2. `0x6c`: As caller, what do we save before making the recursive call? Why?
3. Why do we use the temporary register `t0`?
4. Epilogue: Do we restore registers first or pop stack frame?
5. `0x64`: Why `j epilogue`? Why not `j ra`?

(2/7) Recursive Factorial: C Code

```
0x 0  main:
        li a0 3
   4    jal ra factorial
        …
  48    factorial:
        addi sp sp -8
  4c    sw ra 0(sp)
  50    sw s0 4(sp)
  54    mv s0 a0
  58    li t0 1
  5c    bne s0 t0 recurse
  60    li a0 1
  64    j epilogue
  68    recurse:
        addi a0 s0 -1
  6c    jal ra factorial
  70    mul a0 s0 a0
  74    epilogue:
        lw ra 0(sp)
  78    lw s0 4(sp)
  7c    addi sp sp 8
  80    jr ra
```

Prologue

Epilogue

Set breakpoints:

- factorial(4): `0x4`
- factorial(3): `0x24`
- factorial: `0x48`
  - Base case: `0x64`
  - Recursive call: `0x6c`

Check: registers `a0`, `s0`

```
// assume non-negative numbers
int factorial(int n) {
  if (n == 1) return 1;
  int a = n*factorial(n-1);
  return a;
}
```

(3/7) Recursive Factorial: Question 1

```
0x 0  main:
        li a0 3
   4    jal ra factorial
        …
  48  factorial:
        addi sp sp -8
  4c    sw ra 0(sp)
  50    sw s0 4(sp)
  54    mv s0 a0
  58    li t0 1
  5c    bne s0 t0 recurse
  60    li a0 1
  64    j epilogue
  68  recurse:
        addi a0 s0 -1
  6c    jal ra factorial
  70    mul a0 s0 a0
  74  epilogue:
        lw ra 0(sp)
  78    lw s0 4(sp)
  7c    addi sp sp 8
  80    jr ra
```

Prologue

Epilogue

1. Prologue:
   a. How large is the stack frame?
      - 8B

   b. Which registers get saved to the stack?
      - Our `ra`, because we (may) do a recursive function call
      - Our caller's `s0`

```
// assume non-negative numbers
int factorial(int n) {
  if (n == 1) return 1;
  int a = n*factorial(n-1);
  return a;
}
```

(4/7) Recursive Factorial: Question 2

```
0x 0  main:
        li a0 3
   4    jal ra factorial
        …
  48    factorial:
        addi sp sp -8      } Prologue
  4c    sw ra 0(sp)        }
  50    sw s0 4(sp)        }
  54    mv s0 a0
  58    li t0 1
  5c    bne s0 t0 recurse
  60    li a0 1
  64    j epilogue
  68    recurse:
        addi a0 s0 -1
  6c    jal ra factorial
  70    mul a0 s0 a0
  74    epilogue:
        lw ra 0(sp)        } Epilogue
  78    lw s0 4(sp)        }
  7c    addi sp sp 8       }
  80    jr ra
```

2. `0x6c`: As caller, what do we save before making the recursive call? Why?
   - Copy argument `a0` to saved register `s0`
   - Need to use `s0` after recursive call completes

```
// assume non-negative numbers
int factorial(int n) {
  if (n == 1) return 1;
  int a = n*factorial(n-1);
  return a;
}
```

(5/7) Recursive Factorial: Question 3

```
0x 0  main:
        li a0 3
   4    jal ra factorial
        …
  48    factorial:
        addi sp sp -8        } Prologue
  4c    sw ra 0(sp)          }
  50    sw s0 4(sp)          }
  54    mv s0 a0
  58    li t0 1
  5c    bne s0 t0 recurse
  60    li a0 1
  64    j epilogue
  68    recurse:
        addi a0 s0 -1
  6c    jal ra factorial
  70    mul a0 s0 a0
  74    epilogue:
        lw ra 0(sp)          } Epilogue
  78    lw s0 4(sp)          }
  7c    addi sp sp 8         }
  80    jr ra
```

3. Why do we use the temporary register `t0`?
   - Base case is 1
   - Branch instruction compares two registers

```
// assume non-negative numbers
int factorial(int n) {
  if (n == 1) return 1;
  int a = n*factorial(n-1);
  return a;
}
```

(6/7) Recursive Factorial: Question 4

```
0x 0  main:
        li a0 3
   4    jal ra factorial
        …
  48    factorial:
        addi sp sp -8        } Prologue
  4c    sw ra 0(sp)          }
  50    sw s0 4(sp)          }
  54    mv s0 a0
  58    li t0 1
  5c    bne s0 t0 recurse
  60    li a0 1
  64    j epilogue
  68    recurse:
        addi a0 s0 -1
  6c    jal ra factorial
  70    mul a0 s0 a0
  74    epilogue:
        lw ra 0(sp)          } Epilogue
  78    lw s0 4(sp)          }
  7c    addi sp sp 8         }
  80    jr ra
```

4. Epilogue: Do we restore registers first or pop stack frame?
   - Pop stack frame at the very end, right before returning

```
// assume non-negative numbers
int factorial(int n) {
  if (n == 1) return 1;
  int a = n*factorial(n-1);
  return a;
}
```

(7/7) Recursive Factorial: Question 5

[venus demo]

```
0x 0  main:
        li a0 3
   4    jal ra factorial
   …    …
  48  factorial:
        addi sp sp -8        } Prologue
  4c    sw ra 0(sp)          }
  50    sw s0 4(sp)          }
  54    mv s0 a0
  58    li t0 1
  5c    bne s0 t0 recurse
  60    li a0 1
  64    j epilogue
  68  recurse:
        addi a0 s0 -1
  6c    jal ra factorial
  70    mul a0 s0 a0
  74  epilogue:
        lw ra 0(sp)          } Epilogue
  78    lw s0 4(sp)          }
  7c    addi sp sp 8         }
  80    jr ra
```

5. `0x64`: Why **`j epilogue`**? Why not `j ra`?
   - When returning, **we are always responsible for deallocating our own stack frame!**
   - We must also restore registers:
     - Restore our `ra`, since it may have changed if we made a recursive fn call
     - Restore our caller's `s0`, so they can use it

```
// assume non-negative numbers
int factorial(int n) {
  if (n == 1) return 1;
  int a = n*factorial(n-1);
  return a;
}
```

Agenda

[Reference] Summary, Other Registers

- Function Calls
- Stack Frames
- RISC-V Calling Convention
- [Venus] Recursive Example
- [Reference] Summary, Other Registers
- [Extra] Recursive Example 2

[Summary] RISC-V So Far

- Arithmetic
  - `add`
  - `sub`
  - `and`
  - `or`
  - `xor`
  - `sll`
  - `srl`
  - `sra`
- Immediate
  - `addi`
  - `andi`
  - `ori`
  - `xori`
  - `slli`
  - `srli`
  - `srai`
  - `li` (pseudo)
- Loads/Stores
  - `lw`
  - `lb`
  - `lbu`
  - `sw`
  - `sb`
- Branches/Jumps
  - `beq`
  - `bne`
  - `bge`
  - `blt`
  - `bgeu`
  - `bltu`
  - `j` (pseudo)
  - `jalr`
  - `jal`
  - `jr` (pseudo)

Summary: RISC-V Calling Convention

- Registers faster than memory, so use them!
  - Use stack frames sparingly!
- `a0–a7` (`x10–x17`): Caller-saved
  - eight argument registers to pass parameters
  - two return values (`a0–a1`), though we usually use `a0`
- `ra` (`x1`): Caller-saved
  - one return address register to return to the point of origin
- `sp` (`x2`): Callee-saved
  - stack pointer has address of bottom of stack (current stack frame)
- `s0–s11` (`x8,x9,x18–x27`): Saved registers Callee-saved
- `t0–t6` (`x5–x7,x28–x31`): Temporary registers Caller-saved

Different functions share the **same set of registers** and use them with the **same calling conventions**.

Other Registers

These registers that are out of scope for this class (don't use them!)

- `gp`: The `x3` register, used to store a reference to the heap. Also called the "global pointer"
- `tp`: The `x4` register, used to store separate stacks for threads
  - (multithreading covered way later)

Agenda

[Extra] Recursive Example 2

- Function Calls
- Stack Frames
- RISC-V Calling Convention
- [Venus] Recursive Example
- [Reference] Summary, Other Registers
- [Extra] Recursive Example 2

C code

```
int foo(int i) {
  if (i == 0) return 0;
  int a = i + foo(i-1);
  return a;
}
int j = foo(3);
int k = foo(100);
int m = j+k;
```

RISC-V code

```
j main
foo:            # int foo(int i)
addi sp sp -8   # Prologue
sw ra 0(sp)     # Prologue
sw s0 4(sp)     # Prologue
mv s0 a0        # Move i
bne s0 x0 Next# if i != 0, skip
this
li a0 0         # int a = 0;
j Epilogue      # Go to Epilogue
                # (to restore stack)
```

```
Next:
addi a0 s0 -1 # int j = i - 1;
jal ra foo    # j = foo(j);
add a0 s0 a0  # int a = i + j;
Epilogue:
lw ra 0(sp)   # Epilogue
lw s0 4(sp)   # Epilogue
addi sp sp 8  # Epilogue
jr ra         # return a;
main:
li a0 3       # int j = foo(3);
jal ra foo    # call foo
mv s0 a0      # mv rd rs1 sets rd = rs1
li a0 100     # int k = foo(100);
jal ra foo    # call foo
mv s1 a0      # Saves return value in s1
add a0 s0 s1  # int m = j+k;
```